AFFORDABLE HOUSING AWS S3 DOCUMENT MIGRATION

Engineering Case Study

Field	Description
Author	Venkata Lakshmi Parimala Pasupuleti
Domain	Enterprise Affordable Housing Software
System Type	Document-storage migration and application integration
Primary Focus	Module ownership, document inventory, framework adaptation, validation, QA enablement, and controlled rollout
Status	Production experience documented from reported project participation and ownership
Document Version	1.0

Scope note
This document is intentionally precise about ownership. The organization-wide migration framework was designed by another team. The documented contribution is the Affordable Housing module integration: understanding the module's document types, adapting the shared framework, validating migrated content and metadata, supporting QA, and protecting customer workflows. Broader platform patterns and modernization recommendations are presented as engineering guidance, not as claims of original framework ownership.

Contents

1. Executive Summary
2. Business Context
3. Problem Statement and Goals
4. Ownership and Collaboration Model
5. Requirements
6. Architecture Overview
7. Discovery and Document Inventory
8. Module-Specific Mapping and Framework Adaptation
9. Migration Workflow
10. Data and Metadata Design
11. Validation and Reconciliation
12. QA Enablement and Test Strategy
13. Deployment and Cutover
14. Reliability, Failure Handling, and Recovery
15. Security and Access Control
16. Observability and Operational Evidence
17. Production Operations and Support
18. Design Decisions and Trade-offs
19. Lessons Learned
20. Modernization Direction

Appendix A. Migration Runbook

Appendix B. Validation Checklist

Appendix C. Glossary

1. Executive Summary

The Affordable Housing document migration moved module-specific document assets and their application references into an AWS S3-backed storage model through a shared migration framework. The central engineering challenge was not simply copying files. It was preserving the business identity of each document: what type of document it represented, which customer or property it belonged to, how the application located it, and whether the resulting object could still participate in the same customer workflow after migration.

The organization-wide framework was owned by another team. The Affordable Housing responsibility was to understand the module's document landscape, adapt the framework to its document types and metadata conventions, validate the resulting object and database state, and work with QA to confirm that customer-facing workflows remained functional. This scope required both domain knowledge and integration discipline. A generic migration engine cannot infer module semantics on its own; the module owner must supply the mappings, edge cases, and acceptance criteria that define correctness.

The migration therefore used a layered approach: inventory legacy document records, classify supported document types, map each record into the framework's input contract, migrate the underlying object, update or reconcile the application reference, and perform read-back validation through the expected application path. Records that could not be mapped safely were treated as exceptions rather than guessed into a target representation.

The project reinforced a core senior-engineering principle: shared infrastructure creates leverage, but successful adoption depends on strong module ownership. Reusing a common framework reduced duplicated implementation while preserving clear accountability for module-specific correctness.

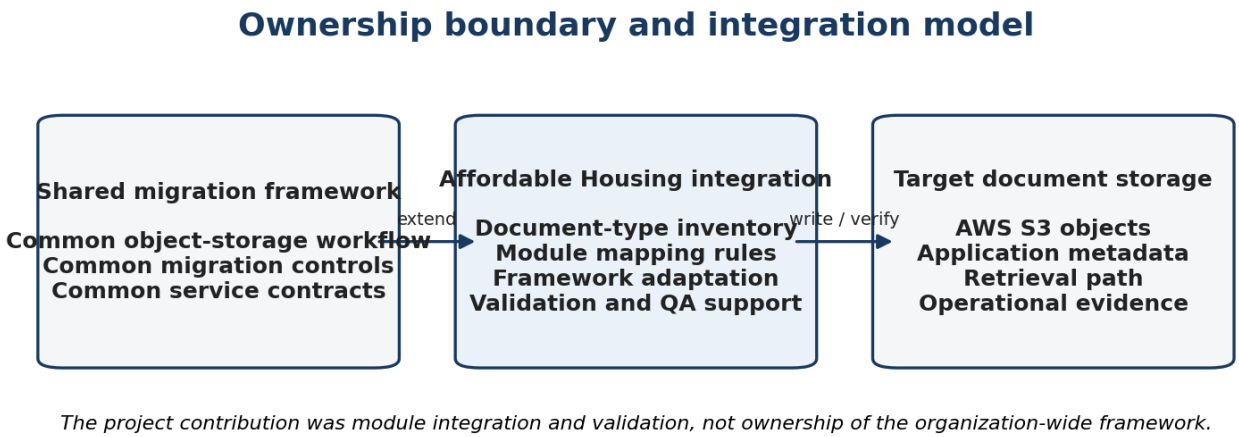


Figure 1. Ownership boundary between the shared migration framework and the Affordable Housing module integration.

2. Business Context

Affordable Housing applications generate and retain documents that support resident, property, certification, compliance, and operational workflows. These documents are not interchangeable binary files. Their meaning is established by the application data surrounding them: document type, property, resident or household association, effective date, generation event, template or source, and retention expectations.

A storage migration can therefore succeed at the infrastructure layer while failing at the business layer. An object may exist in S3, but the migration is still incorrect if the application cannot find it, the wrong customer sees it, the document type is misclassified, or the metadata no longer supports downstream workflows. The module integration had to preserve both content and context.

2.1 Why document storage is business critical

- Documents are customer-visible outputs and may be used in regulated or audited workflows.
- Historical records must remain retrievable after an infrastructure change.
- Document metadata controls ownership, organization, access, and application behavior.
- A missing or mismatched document can become a support incident even when the storage service itself is healthy.
- Migration defects may remain hidden until a user opens an older property or resident record.

2.2 Business objective

Move the Affordable Housing module onto the organization's S3-backed document-storage path without breaking document generation, retrieval, historical access, or customer-facing workflows.

3. Problem Statement and Goals

The core problem was to integrate a domain-specific document model with a shared storage-migration framework. The framework provided a standard mechanism for moving and registering documents, but it required each product module to define how its legacy records mapped to the framework contract.

The Affordable Housing module contained multiple document types and historical behaviors. The integration needed to distinguish valid in-scope documents from unsupported or ambiguous records, preserve the correct application relationships, and create sufficient evidence that the migration had not silently changed business meaning.

3.1 System goals

- Identify the complete set of Affordable Housing document types within migration scope.
- Map legacy records into the shared framework without losing business identity.
- Preserve the relationship among document content, metadata, and owning entity.
- Validate both object storage and application-level retrieval.
- Provide QA with scenarios, expected results, and traceable evidence.
- Capture exceptions explicitly so uncertain records do not become silent data corruption.
- Support a controlled release with clear stop and recovery conditions.

3.2 Non-goals

- Redesigning the organization-wide migration framework.
- Replacing the Affordable Housing document-generation engine.

- Changing document business meaning during the storage move.
- Using the migration to perform unrelated schema or product refactoring.
- Assuming every historical record follows the current data model.

4. Ownership and Collaboration Model

The project used a shared ownership model. The platform or migration team owned the common framework, target-storage integration, and reusable migration mechanics. The Affordable Housing engineer owned the module-specific interpretation needed to use that framework safely.

Responsibility Area	Primary Owner	Engineering Accountability
Shared migration framework	Platform / migration team	Common execution path, storage contract, reusable controls, and framework defects
Affordable Housing document inventory	Affordable Housing engineer	Identify document categories, legacy sources, and module-specific edge cases
Mapping and adapter logic	Affordable Housing engineer with framework collaboration	Translate module records into the shared migration contract
Database and infrastructure support	Database / platform teams	Backups, access, environment support, and production-safe execution
Functional validation	Affordable Housing engineering and QA	Verify retrieval, metadata, ownership, and user workflows
Release coordination	Cross-functional release team	Sequence, communication, stop conditions, and incident handling

Ownership principle
The framework owner is responsible for generic mechanics. The module owner is responsible for semantic correctness. Neither role can safely replace the other.

5. Requirements

5.1 Functional requirements

- Read legacy Affordable Housing document records and locate their underlying content.
- Classify each record into a supported target document type.
- Create the target S3 object through the approved shared framework.
- Preserve or reconstruct required metadata and application ownership references.
- Allow the application to retrieve migrated documents through the normal workflow.
- Record migration status and exception details for reconciliation.
- Support rerun or recovery behavior without uncontrolled duplication.

5.2 Non-functional requirements

- Correctness: content and business identity must remain aligned.
- Auditability: every source record must have an explainable outcome.
- Reliability: transient failures must not cause silent loss or inconsistent state.
- Security: object access must follow the application's authorization model.
- Operational control: migration execution must be observable and stoppable.
- Maintainability: module-specific mappings should remain isolated from generic framework logic.
- Performance: throughput should be sufficient for the migration window without destabilizing the application or database.

6. Architecture Overview

The migration architecture separates module interpretation from generic storage mechanics. Affordable Housing code discovers and classifies source records, builds the framework input contract, and supplies the metadata required to preserve business meaning. The shared framework performs the standard migration sequence and writes to the S3-backed storage layer. Validation then reconciles the source, target object, metadata, and application read path.

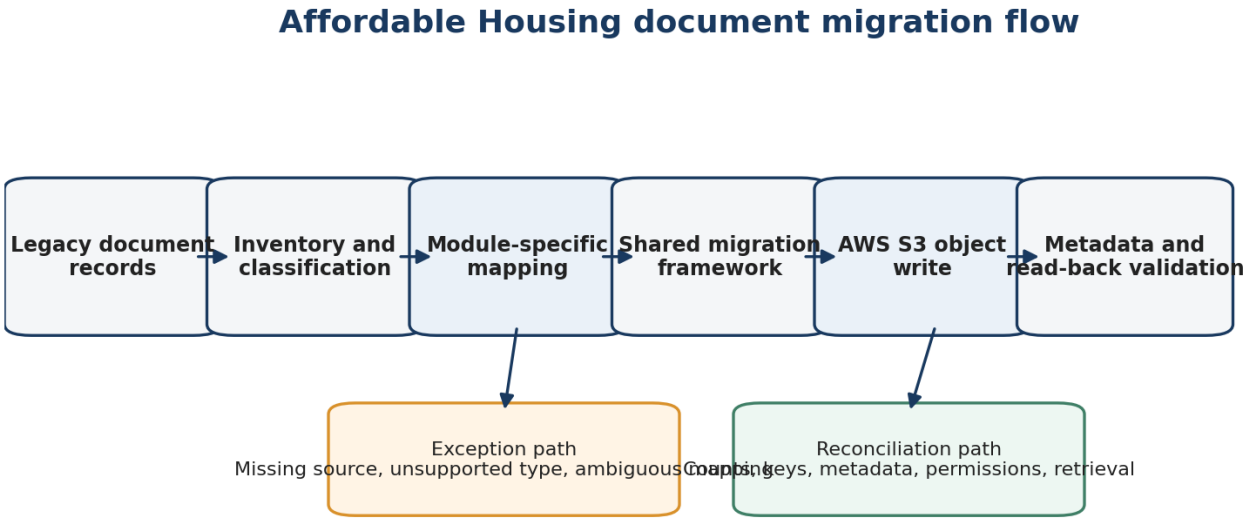


Figure 2. End-to-end Affordable Housing document migration flow.

6.1 Logical components

Component	Responsibility
Legacy document repository	Holds source document content and legacy references.
Affordable Housing inventory query	Finds in-scope records and gathers module-specific context.
Classification and mapping adapter	Converts legacy type and ownership conventions into the framework contract.
Shared migration framework	Coordinates object transfer, target registration, and common status handling.
AWS S3 storage	Stores target document objects under the platform's approved storage model.
Application metadata store	Retains the relationship between the application entity and target object.
Validation and reconciliation process	Proves that the migrated object remains usable and correctly associated.

6.2 Separation of concerns

Keeping the module adapter separate from the shared framework avoids embedding Affordable Housing rules into generic infrastructure. It also allows framework improvements to benefit multiple modules without rewriting domain mappings. The trade-off is an integration contract that must be precise and versioned carefully.

7. Discovery and Document Inventory

Discovery was the first correctness control. Before migration logic could be trusted, the team needed to understand what the Affordable Housing module actually stored, how document records were identified, and which historical exceptions existed. The inventory was not simply a total row count; it was a classification of the document population by business meaning and technical shape.

7.1 Inventory dimensions

- Document type and subtype.
- Legacy storage location or source mechanism.
- Owning property, resident, household, certification, or workflow entity.
- Creation or effective date and historical version behavior.
- Presence and validity of source content.
- Availability of required target metadata.
- Known duplicates, orphaned references, or unsupported historical formats.

7.2 Classification outcomes

Outcome	Meaning	Required Action
Supported	Source record has a known document type, valid content, and complete mapping inputs.	Migrate through the standard path.
Conditionally supported	Record is valid but requires a module-specific mapping or data repair.	Apply reviewed transformation and validate explicitly.
Excluded	Record is intentionally outside the migration scope.	Document the exclusion reason and reconcile counts.
Failed	Source content or required metadata is unavailable.	Capture as an exception; do not infer a target state.
Ambiguous	More than one target mapping is plausible.	Escalate for domain decision before migration.

Discovery lesson
Historical enterprise data often reflects multiple generations of application behavior. Current code paths are not a complete specification of historical storage.

8. Module-Specific Mapping and Framework Adaptation

The central implementation responsibility was adapting Affordable Housing document records to the framework contract. This required explicit mapping logic rather than relying on naming conventions or broad assumptions. A correct adapter translates both technical fields and domain meaning.

8.1 Mapping contract

Input Category	Examples of Required Meaning	Validation
Source identity	Legacy record identifier and content location	Must resolve to exactly one source object or explicit exception.
Target document type	Approved platform classification	Must be a deterministic mapping from the legacy type and context.
Owner context	Property, resident, household, certification, or workflow owner	Must match application authorization and retrieval behavior.
Object metadata	File name, media type, timestamps, module tags	Must preserve required search, display, and audit semantics.
Target key inputs	Tenant or account scope, entity identifiers, generated identity	Must avoid collisions and cross-customer ambiguity.
Migration evidence	Status, timestamps, errors, source-to-target link	Must support reconciliation and rerun decisions.

8.2 Adapter design principles

- Use explicit mapping tables or functions for document types rather than fall-through defaults.
- Fail closed when the adapter cannot establish a safe target representation.
- Keep module-specific rules outside the shared framework core.
- Make mapping outcomes deterministic for the same source state.
- Include enough source and target identifiers in logs to support reconciliation without exposing sensitive content.
- Treat historical exceptions as first-class cases, not as noise to be ignored.

8.3 Why not duplicate the framework?

Building a separate Affordable Housing migration engine would have increased implementation speed only temporarily. It would also duplicate storage logic, operational controls, and future maintenance. Reusing the shared framework preserved organizational consistency while allowing the module team to focus on business-specific correctness.

9. Migration Workflow

The migration workflow was organized as a sequence of controlled, evidence-producing stages. Each stage reduced uncertainty before the next stage increased blast radius.

1. Discover and count candidate records by document type and ownership category.
2. Classify records as supported, conditional, excluded, failed, or ambiguous.
3. Build the module-specific framework input for supported records.
4. Execute the shared migration operation and capture the target object identity.
5. Update or reconcile the application metadata reference as defined by the framework.
6. Perform object-level and metadata-level validation.
7. Read the document back through the expected application path.
8. Reconcile totals and exception outcomes before approving the next batch or cutover.

9.1 Rerun and idempotency considerations

A migration process must anticipate partial execution. If object creation succeeds but metadata registration fails, a blind rerun can create duplicates. If metadata changes but object creation fails, the application can point to a missing object. The safest design records stage-level state and uses a stable source identity or migration key to determine whether a record is new, completed, recoverable, or inconsistent.

Observed State	Safe Response
No target evidence exists	Execute the full migration path.
Target object exists and metadata is valid	Treat as completed; do not duplicate.
Target object exists but metadata is missing	Repair or resume registration after validating object identity.
Metadata points to a missing object	Stop and investigate; do not treat the row as migrated.
Multiple targets exist for one source	Quarantine the record and reconcile manually or through an approved repair path.

10. Data and Metadata Design

The object itself is only one part of the document record. The application also needs a stable metadata relationship so it can authorize, organize, display, and retrieve the object. The migration must keep these layers aligned.

10.1 Conceptual target record

Field Group	Purpose
Migration identity	Links the legacy record to the target record and supports reconciliation.
Object identity	Stores the S3 bucket or storage namespace and object key or platform object identifier.
Business ownership	Identifies the tenant, property, resident, household, certification, or workflow context.
Document classification	Represents the product-defined type and subtype.
Content properties	File name, media type, size, and optional integrity value.
Lifecycle metadata	Created, migrated, superseded, retained, or deleted status as supported by the platform.
Audit metadata	Execution identifier, timestamps, operator or service identity, and failure reason.

10.2 Object-key design considerations

The exact S3 key convention belonged to the shared framework. From the module integration perspective, the key requirements were uniqueness, tenant isolation, stable traceability, and no dependence on mutable display values. A display name such as a property name can change; an immutable identifier is safer as a storage-key component.

10.3 Metadata consistency

The application metadata and object storage state form a distributed consistency problem. When they cannot be committed in one atomic transaction, the workflow must use explicit status transitions, compensation, or reconciliation. Operational evidence must make an incomplete state detectable rather than invisible.

11. Validation and Reconciliation

Validation was the most important part of the module integration. A migration result was not accepted simply because the framework completed without an exception. Validation moved from population-level reconciliation to application-level read-back.

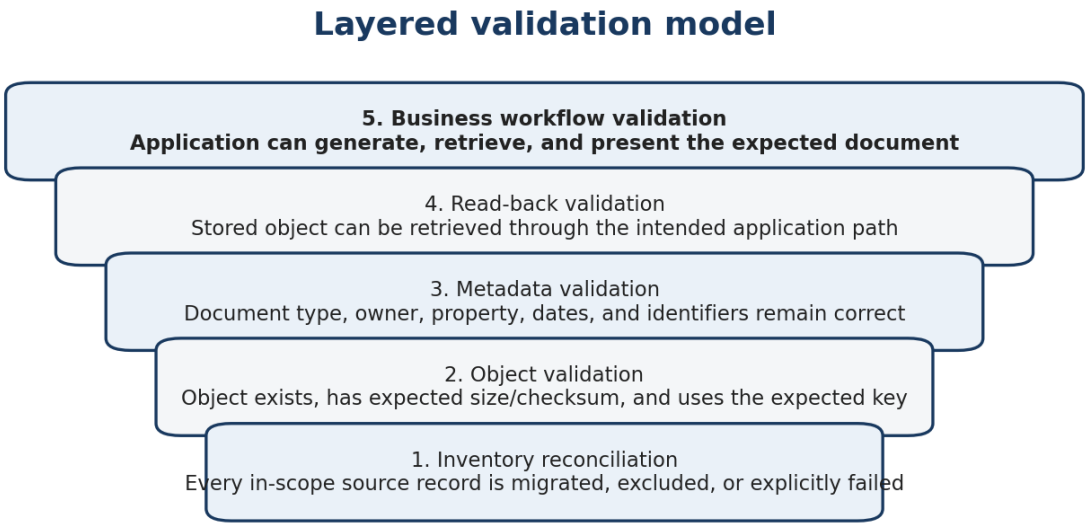


Figure 3. Layered validation from source inventory to customer-facing workflow.

11.1 Population reconciliation

Every in-scope source record needed one explainable outcome: migrated successfully, excluded by approved scope, or failed with a documented reason. This prevents a small set of missing documents from disappearing inside a large success count.

11.2 Object validation

- Target object exists under the expected identity.
- Object size is plausible relative to the source.
- Content can be opened or parsed as the expected file type.
- Optional integrity checks match when source and target checksums are available.
- No unexpected duplicate target is created for the same source.

11.3 Metadata validation

- Document type and subtype are correct.
- Property and customer ownership remain correct.
- Related entity identifiers still resolve.
- Creation and effective dates retain required meaning.
- Application display and retrieval behavior match the pre-migration expectation.

11.4 Application read-back

The strongest validation is not a direct S3 lookup; it is retrieval through the same application service or user workflow that will be used after cutover. This verifies storage, metadata, authorization, and application integration together.

12. QA Enablement and Test Strategy

The Affordable Housing engineer supported QA by translating migration mechanics into domain scenarios. QA needed more than a list of object keys. It needed representative documents, ownership relationships, historical cases, and expected customer behavior.

12.1 Test coverage model

Test Category	Representative Coverage
Document-type coverage	At least one valid case for every supported Affordable Housing document type and important subtype.
Ownership coverage	Documents linked to different entity types and customer/property contexts.
Historical coverage	Older documents created under previous application behaviors or legacy formats.
Negative coverage	Missing content, unsupported type, incomplete metadata, and invalid ownership.
Rerun coverage	Already migrated, partially migrated, and failed records.
User workflow coverage	Generate, list, open, download, and retrieve historical documents through the application.

12.2 Acceptance criteria

- No supported source record is missing from reconciliation.
- Migrated documents remain accessible only to the correct application scope.
- Representative historical documents open successfully.
- New document generation continues to write and retrieve through the target path.
- Failures are visible with enough context for engineering triage.
- Rollback or stop conditions are understood before production execution.

QA collaboration lesson
A module owner improves migration quality by giving QA a domain-oriented test matrix, not by asking QA to infer business significance from raw storage records.

13. Deployment and Cutover

A controlled rollout reduces the risk of discovering a module-specific mapping problem after the entire document population has moved. The migration should progress through environments and bounded batches with validation checkpoints.

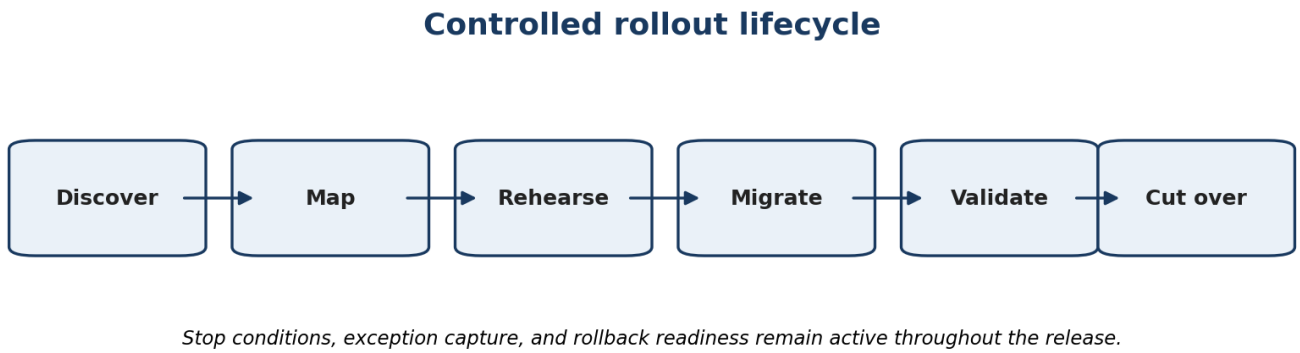


Figure 4. Controlled rollout lifecycle with stop conditions at every stage.

13.1 Recommended execution sequence

9. Validate framework compatibility and module mappings in a development environment.
10. Rehearse against representative or production-like data in a non-production environment.
11. Run inventory and dry-run reports before any target write.
12. Migrate a small, representative canary set.
13. Perform layered validation and application read-back.
14. Expand batch size only after reconciliation passes.
15. Complete cutover or enable the target read path according to the platform release plan.
16. Continue post-cutover monitoring and historical sampling.

13.2 Stop conditions

- Unexpected growth in failed or ambiguous mappings.
- Count reconciliation does not balance.
- Objects exist but application read-back fails.
- Ownership or authorization is incorrect.
- Rerun creates duplicate targets.
- Database or storage load threatens production stability.
- A framework behavior differs from the validated non-production contract.

14. Reliability, Failure Handling, and Recovery

Document migration spans at least two stateful systems: application metadata and object storage. Reliability depends on making every intermediate state observable and recoverable.

14.1 Failure scenarios

Failure Scenario	Risk	Control
Source content missing	Record appears valid but cannot be migrated.	Mark failed with source identity; do not create an empty target.
Unsupported document type	Incorrect generic classification.	Fail closed and require an approved mapping.
S3 write fails transiently	Incomplete batch and repeated work.	Retry through framework policy and preserve stage status.
Object succeeds, metadata fails	Orphan target object.	Resume or compensate using the stable migration identity.
Metadata succeeds, object missing	Broken application link.	Block completion and reconcile before cutover.
Incorrect owner mapping	Cross-customer exposure or inaccessible document.	Validate ownership and authorization through application read-back.
Duplicate execution	Multiple objects for one source.	Use idempotency or uniqueness controls based on source identity.
Historical format cannot be parsed	Silent loss or unreadable object.	Quarantine, preserve source, and route for domain review.

14.2 Recovery model

Recovery should operate at the record or bounded-batch level where possible. A full rollback may be unnecessary if the system can identify incomplete records, preserve valid completed work, and safely resume. However, cutover must not occur until the application read path is consistent for the intended population.

14.3 Source preservation

The original document source should remain available for the approved recovery window. Removing the source immediately after a successful copy eliminates the strongest evidence available when later discrepancies are discovered.

15. Security and Access Control

Moving documents to S3 changes the infrastructure boundary but must not weaken the product authorization boundary. The application should remain responsible for deciding which authenticated user can access a document. Direct object access should follow the platform's approved mechanism rather than exposing broad bucket permissions.

15.1 Security controls

- Tenant and customer isolation in metadata and object identity.
- Least-privilege service permissions for migration and application access.
- Encryption in transit and platform-approved encryption at rest.
- No sensitive document content in migration logs.
- Auditable service identity for migration actions.
- Controlled handling of temporary files or staging locations.
- Validation that object access cannot bypass application authorization.

Security boundary
A successful object write is not proof of secure integration. Security is validated only when the application can retrieve the object for an authorized user and deny it outside the correct scope.

16. Observability and Operational Evidence

Migration observability should answer four questions: What was selected? What happened to each record? What is the target identity? Can the application use it? A single aggregate success count is not sufficient.

16.1 Minimum evidence per record

- Source record and document identity.
- Migration execution or batch identifier.
- Classification and mapping outcome.
- Target object or platform document identity.
- Start and completion timestamps.
- Final status and failure category.
- Retry or resume history where applicable.
- Validation status and application read-back result.

16.2 Operational metrics

- Candidates discovered - establishes the source population.
- Classification outcomes - explains supported, conditional, excluded, failed, and ambiguous records.
- Objects written and bytes transferred - tracks progress and throughput.
- Framework failures by category - separates infrastructure failures from mapping defects.
- Validation failures by layer - identifies inventory, object, metadata, or application issues.
- Duplicate or inconsistent-state count - detects rerun and partial-commit problems.
- Application read-back success rate - measures customer-visible readiness.

17. Production Operations and Support

Post-release support focuses on the customer workflow rather than only the migration job. A support issue may surface as a missing historical document, an incorrect document type, a permission error, or an application record that no longer resolves to its content.

17.1 Triage sequence

17. Identify the customer, property, document type, and business workflow.
18. Locate the legacy source identity and migration evidence.
19. Confirm the target object exists and matches the expected migration identity.
20. Validate the metadata ownership and document classification.
21. Reproduce retrieval through the normal application path.
22. Determine whether the issue is content, mapping, authorization, framework behavior, or application integration.
23. Apply an approved repair or recovery path and update the incident record.

17.2 Support evidence packet

- Customer and property context.
- Source and target document identifiers.
- Document type and expected workflow.
- Migration execution and status.
- Object and metadata validation results.
- Application error or reproduction steps.
- Known similar exceptions and approved repair guidance.

18. Design Decisions and Trade-offs

Decision	Benefit	Trade-off
Reuse the shared framework	Common controls and less duplicated infrastructure.	Requires a precise cross-team contract.
Use explicit module mappings	Auditable handling of historical differences.	Mappings require ongoing maintenance.
Fail closed on ambiguity	Prevents silent ownership or type errors.	Creates cases for domain review.
Validate through the application	Tests customer-visible integration.	Costs more than direct object checks.
Use bounded batches	Limits blast radius and supports learning.	Extends elapsed migration time.
Preserve the source temporarily	Supports recovery and investigation.	Creates temporary duplicate storage.
Keep dedicated migration evidence	Improves reconciliation and auditability.	Requires additional operational tooling.

18.1 Why shared infrastructure was the right boundary

The platform team had the strongest context for reusable S3 integration and common migration controls. The Affordable Housing team had the strongest context for document meaning and customer behavior. The design aligned ownership with knowledge rather than forcing one team to own both domains.

19. Lessons Learned

- A storage migration is a business-data migration, not a file-copy task.
- Module-specific knowledge is essential even when a reusable platform exists.
- Inventory and classification are implementation work, not preliminary paperwork.
- Every source record needs an explainable final outcome.
- Application read-back is stronger evidence than an object-store success response.
- Historical data requires explicit exception handling because current models do not describe every legacy state.
- QA becomes more effective when engineering provides a domain-based test matrix.
- Accurate ownership statements strengthen credibility: integration ownership is meaningful even when another team owns the shared framework.
- Operational evidence should be designed before production execution, not assembled after a failure.

20. Modernization Direction

Status distinction
The following items describe a modern target architecture and operational improvements. They are not claims about the original implementation unless explicitly stated.

20.1 Manifest-driven migration

Generate a versioned manifest that records the complete source population, expected mapping, target identity, and validation policy before execution. The manifest becomes the contract for reconciliation and repeatability.

20.2 Durable workflow orchestration

For large or long-running migrations, use a durable workflow or queue-based execution model that records stage transitions, supports bounded concurrency, and resumes safely after worker failure. This should be introduced because of workload and recovery requirements, not simply because asynchronous infrastructure is available.

20.3 Automated integrity and read-back checks

Automate content-integrity checks where source and target hashes are available, and run application-service read-back probes against representative migrated records after each batch.

20.4 Schema-enforced mapping contracts

Define module adapter inputs and outputs using a versioned schema. Contract tests should fail when framework or module changes break the migration interface.

20.5 Observability dashboard

Expose batch progress, classification outcomes, failure categories, validation layers, and unresolved exceptions in one operational dashboard. Operators should be able to move from a summary metric to an individual source-to-target trace.

20.6 Security automation

Continuously verify bucket policies, service permissions, encryption settings, and tenant-isolation assumptions through automated policy checks and application authorization tests.

Appendix A. Migration Runbook

A.1 Before execution

- Confirm approved scope, document types, and exclusions.
- Capture source inventory counts and exception categories.
- Validate adapter and framework versions.
- Confirm database backup or recovery readiness owned by the appropriate team.
- Validate target permissions and storage configuration.

- Review canary selection and acceptance criteria with QA.
- Confirm release communication, operators, and stop authority.

A.2 During execution

- Record execution and batch identifiers.
- Monitor throughput, errors, and unresolved mappings.
- Stop on ownership, authorization, reconciliation, or application read-back failures.
- Validate canary and each bounded batch before expansion.
- Preserve logs and source-to-target evidence.

A.3 After execution

- Reconcile every source record to a final outcome.
- Run application read-back and historical sampling.
- Confirm new document generation and retrieval use the intended path.
- Review unresolved exceptions and assign owners.
- Monitor support signals through the post-cutover window.
- Close the migration only after evidence and recovery requirements are satisfied.

Appendix B. Validation Checklist

Validation Area	Checklist
Inventory	Counts by document type and owner; no unexplained population gaps.
Mapping	Every supported type has an explicit, reviewed target mapping.
Object	Exists, expected size/type, optional integrity value, no uncontrolled duplicate.
Metadata	Correct type, customer/property ownership, related entity, dates, and status.
Application	List, open, download, and historical retrieval succeed through the normal path.
Security	Authorized scope succeeds; unauthorized scope is denied.
Recovery	Partial states are detectable and have an approved resume or repair path.
Operations	Execution, error, and validation evidence are retained and searchable.

Appendix C. Glossary

Term	Definition
Adapter	Module-specific code that translates Affordable Housing document records into the shared framework contract.
Application read-back	Retrieving a migrated document through the normal application service or user workflow.
Framework	The organization-wide reusable migration mechanism owned by another team.
Manifest	A versioned representation of the source population and expected migration outcomes.
Object key	The target storage identity used by S3 or an abstraction over it.
Reconciliation	Proving that all source records have a correct, explainable final outcome.
Source identity	Stable identifier used to trace a legacy record through migration and rerun handling.
Target metadata	Application data that connects the S3 object to its document type and business owner.